



Last chance to fix `std::nontype`

Document number:

[P3740R1](#)

Date:

2025-06-20

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

Co-authors:

Bronek Kozicki <brok@incorrekt.com>

GitHub Issue:

wg21.link/P3740/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/nontype.cow



ABSTRACT: P2472R3 introduced overloads for the `std::function_ref` constructor which utilize a helper type `std::nontype`. This helper type may be obsoleted by other active proposals, and its name has become a bad choice now that C++26 no longer replaced the term "non-type template parameter", with "constant template parameter".

§ Contents

- 1 **Revision history**
 - 1.1 Changes since R0
- 2 **Introduction**
 - 2.1 Why did we need a new `std::nontype` type?
 - 2.2 Refresher on what `std::nontype` accomplishes
- 3 **Options considered**
 - 3.1 Using a `std::stateless` constructor tag instead
 - 3.2 Removing `std::nontype` with no replacement, revisiting for C++29
 - 3.3 Waiting for `constexpr` function parameters
 - 3.4 Replacing `std::nontype` with `std::constant_wrapper`
 - 3.5 Exposing the *BoundEntityType*



3.6	Renaming <code>std::nontype</code>
3.7	Conclusion
4	Implementation experience
5	Wording
5.1	Option A — Replacing <code>std::nontype</code> with <code>std::constant_wrapper</code>
5.1.1	[version.syn]
5.1.2	[utility.syn]
5.1.3	[func.wrap.func] <code>std::function</code>
5.1.4	[func.wrap.move] <code>std::move_only_function</code>
5.1.5	[func.wrap.copy] <code>std::copyable_function</code>
5.1.6	[func.wrap.ref] <code>std::function_ref</code>
5.2	Option B — Renaming <code>std::nontype</code> to <code>std::constant_arg</code>
6	References

§ 1. Revision history

§ 1.1. Changes since R0

- abandoned the use of a `std::stateless` constructor tag, as it became apparent that this approach has little chance of finding consensus
- instead investigated feasibility of using `std::constant_wrapper` instead of `std::nontype`, with proposed wording for that approach

§ 2. Introduction

[P2472R3] proposed additional overloads for the `std::function_ref` constructor which utilize a helper type `std::nontype`. This paper was merged into [P0792R14], which was plenary-approved at Varna 2023 for C++26.

There is now a proposal [P2781R8] on track for C++29 which adds a `std::constant_wrapper` helper type, and this could be a plausible replacement for `std::nontype`. Considering that, do we even need `std::nontype`? Furthermore, the naming choice `std::nontype` makes very little sense following [P2841R1], which introduced concept and variable template parameters. Since those are not types either, what the standard used to call "non-type template parameter" has been renamed to "constant template parameter".

In short, we now have to decide *for* C++26 whether `std::nontype` should be renamed (and if so, what to), whether it should be merged with `std::constant_wrapper`, or whether these constructor overloads should be delayed until C++29.



NOTE: The class template is called `std::nontype_t`, and the corresponding variable template is called `std::nontype`. This document refers to the feature as a whole as



§ 2.1. Why did we need a new std::nontype type?

An obvious question may be why the existing std::integral_constant cannot be used instead. This has multiple reasons:

- std::integral_constant<class T, T v> requires specifying the type separately.
- std::integral_constant has a lengthy name.
- std::integral_constant has a call operator which returns v, which produces friction with the other constructors.

§ 2.2. Refresher on what std::nontype accomplishes

std::nontype is used only within constructors of std::function_ref:

```
// constexpr and noexcept omitted for simplicity
template<class F>      function_ref(F*);
template<class F>      function_ref(F&&);
template<auto f>       function_ref(nontype_t<f>);
template<auto f, class U> function_ref(nontype_t<f>, U&&);
template<auto f, class T> function_ref(nontype_t<f>, cv T*);
```

Intuitively, std::function_ref is the C++ counterpart to the C idiom of passing void* and a function pointer which that void* is fed into, as seen in qsort. It is common practice to provide a null pointer to qsort and thus rely on a "capture-less" comparison.

With std::nontype, std::function_ref can support such use "capture-less" uses:

```
// once again, some simplifications (no noexcept, no invoke, etc.) ...
template<auto f>
function_ref(nontype_t<f>) {
    // bound-entity is similar to the void* we would have stored in C.
    // We can leave it empty since we are constructing from some global callable.
    this->bound-entity = {};
    // Note that lambdas with no captures can be converted to function pointers.
    this->thunk_ptr = [](BoundEntityType, Args&&... args) -> R {
        // Notice that we don't need to capture f.
        // This is only possible because f is a constant template parameter.
        return f(std::forward<Args&&>(args)...);
    };
}
```

Crucially, it would be impossible to create a std::function_ref from a function directly without this helper. At best, we could store function pointer within bound-entity and call that function pointer from within bound-entity. This would introduce entirely unnecessary overhead.

Besides overhead, we need std::nontype constructors to make a std::function_ref which has the functionality provided by a lambda, but does not reference any specific lambda. This is crucial

for being able to store `std::function_ref` somewhere long-term without paying attention to the lifetime of the callable type it has been initialized with.



§ 3. Options considered

Besides the proposed approach, there are other possible solutions. However, the author position is that every one of them is worse than what is proposed.

§ 3.1. Using a `std::stateless` constructor tag instead



IMPORTANT: This approach was the author's proposed solution in R0. Despite not polling in LEWG on this issue, it was readily apparent that the room had no appetite for this change in direction. In particular, the `std::nontype` technique is less verbose at the call site, which convinced multiple attendees that it would be the superior approach.

A more detailed discussion and proposed wording for this approach can be found in R0 of this paper.

We can ditch `std::nontype` and obtain the functionality it provides in a different way. This would be very tempting because we don't have the dilemma of naming it right, making it redundant with `std::constant_wrapper`, having a weird workaround that becomes obsolete with `constexpr` function parameters, etc. Instead, we can get the same functionality using a tagged constructor.

The crucial observation in §2.2. [Refresher on what `std::nontype` accomplishes](#) is that the lambda we create for `thunk_ptr` cannot capture the constructor parameters. However, that is not actually necessary if we only need the type of the parameter, not its identity. The same approach is used for `std::less`, `std::allocator`, `std::hash`, etc. We don't need to hold an instance of these; we can just default-construct when needed. Specifically, the tagged constructor would function like:

```
// Helper tag type.
struct stateless_t { };
inline constexpr stateless_t stateless;

// once again, some simplifications (no noexcept, no invoke, etc.) ...
template<class F, class T>
function_ref(stateless_t, const F&, cv T* obj) {
    // In practice, this may be slightly more complicated, if BoundEntityType is not
    this->bound-entity = obj;

    this->thunk_ptr = [](BoundEntityType bound, Args&&... args) -> R {
        auto obj = static_cast<cv T*>(bound);
        // No captures needed because we construct a new T here.
        return F{}(obj, std::forward<Args&&>(args)...);
    };
}

// Same idea, but bound-entity becomes null:
template<class F> function_ref(stateless_t, const F&);
```

```
// Same idea, but bound-entity becomes addressof(obj):
template<class F, class U> function_ref(stateless_t, U&& obj);
```



§ 3.2. Removing `std::nontype` with no replacement, revisiting for C++29

Since there is a lot of active work in this area, perhaps we could simply shove `std::nontype` into C++29 and deal with the problem later. However, as demonstrated in §2.2. Refresher on what `std::nontype` accomplishes, this helper type covers crucial use cases such as

- constructing `std::function_ref` from a free function with no overhead, or
- constructing `std::function_ref` from an existing function pointer and `void*`, in the style of `qsort`.

Author position: The functionality provided by `std::nontype` is a crucial part of `std::function_ref`, not a nice-to-have feature that can be ripped out. `std::function_ref` without these constructors would be akin to using containers without move semantics.

§ 3.3. Waiting for `constexpr` function parameters

`std::integral_constant`, `std::constant_wrapper`, and `std::nontype` are – to an extent – temporary hacks. If we were able to write

```
function_ref constexpr F*);
```

... then the workaround of `std::nontype` would be obsolete.

At Kona 2023, EWG showed enthusiasm for this option when discussing [P1045R1] “`constexpr` Function parameters”.

”

Poll: P2781R3 “`std::constexpr_v`” and P1045R1 “`constexpr` function parameters” EWG would like to solve the problem solved by `std::constexpr_v` in the language, for example as proposed by P1045R1 or with “expression aliases”, rather than solving it in library. An implementation is desired. C++26 seems ambitious.

SF	F	N	A	SA
6	8	5	1	0

However, [P1045R1] has not seen any activity from the author since 2019. As nice as a core language solution would be, standard library features cannot be built on hopes, dreams, and unimplemented hypotheticals. Even if `constexpr` function parameters were eventually implemented, it may be possible to integrate them into the existing set of constructors and to deprecate `std::nontype`.

Author position: Overall, there is little reason to treat `constexpr` function parameters as a prerequisite; we don't need to wait.

§ 3.4. Replacing `std::nontype` with `std::constant_wrapper`



The seemingly obvious solution is to use `std::constant_wrapper` from [P2781R8]. The option to use `std::constant_wrapper` as a replacement for `std::nontype` was discussed during an LEWG telecon 2025-03-11. Major concerns have been raised at the time, but after gaining implementation experience, these have been addressed. In particular:

- `std::constant_wrapper` already has a call operator (which returns a `std::constant_wrapper`) and is convertible to its wrapped constant, so this was suspected to cause issues in the constructor overload set or otherwise. However, the other overloads taking `std::nontype` (now `std::constant_wrapper`) would always win in overload resolution, so the extra functionality is inconsequential.
- `std::constant_wrapper` could be used with other function wrappers (`std::move_only_function` etc.) in some cases already (see below), and its behavior would be inconsistent with `std::function_ref`. We address this by adding corresponding overloads to other function wrapper constructors.



EXAMPLE: To illustrate the second point, the following code would compile without `std::constant_wrapper` being treated specially by `std::move_only_function`.

```
constexpr int f() { return 0; }
std::move_only_function<int()> r = std::cw<f>;
```

This works because the type of `std::cw<f>` has a call operator which returns `std::cw<0>` in this case, which is convertible to `int`.

Author position: Considering that `std::nontype` is intended to wrap a constant, much like `std::constant_wrapper`. If this can be done with no major technical issues, it presumably *should* be done. However, if we do this, we have to add corresponding support for `std::constant_wrapper` for other function wrappers so that the use of `std::constant_wrapper` is not subtly different for different wrappers. Also, this can only be done now; adding support for that in C++29 risks altering the meaning of existing C++26 code which uses `std::constant_wrapper`.

§ 3.5. Exposing the *BoundEntityType*

Remember the example in §2.2. [Refresher on what `std::nontype` accomplishes](#). Our end goal is to enable more possible constructions of `std::function_ref`. Technically, this could be done by ripping the implementation open and exposing its details:

```
// new constructor
function_ref(R(*thunk)(BoundEntityType, Args&&...), BoundEntityType entity = { })
: thunk_ptr(thunk)
, bound-entity(entity) { }
```

However, this means that the user directly interacts with *BoundEntityType*, and this is a detail which may vary from implementation to implementation. For implementations that support

conversion between function pointers and `void*`, `BoundEntityType` could simply be `cv void*`, but this may also be a `union`, it may be multiple pointers large, etc.



Author position: Exposing the `BoundEntityType` to the user seems like a bad idea, and it would encourage writing code that is inadvertently not portable.

§ 3.6. Renaming `std::nontype`

Perhaps the most simple solution is to simply rename `std::nontype`. However, no good candidate with high consensus has been presented yet, and many options would be awfully similar to `std::constant_wrapper`, creating confusion:

- `std::constant` seems almost like a "more official" alternative to `std::constant_wrapper`, when this type is extremely limited on scope.
- `std::ctp`, `std::cta`, `std::ct` and other abbreviations derived from "constant template argument/parameter" feel out-of-place in the standard library, and cannibalize `std::cw`, the variable template for `std::constant_wrapper`.
- `std::const_arg`, `std::constant_arg`, `std::constexpr_arg` and other such names describe the purpose of the type well, but are still reminiscent of `std::constant_wrapper`.

Author position: Renaming `std::nontype` is not trivial, and it's unlikely that any solution will make everyone happy. Of all the options, `std::constant_arg` seems like the least bad candidate.

§ 3.7. Conclusion

Every alternative seems bad in its own way, but §3.4. Replacing `std::nontype` with `std::constant_wrapper` seems most obvious, and §3.6. Renaming `std::nontype` seems least intrusive. These are definitely something that we could do for C++26.



NOTE: In §5. Wording, both the wording for §3.4. Replacing `std::nontype` with `std::constant_wrapper` and for §3.6. Renaming `std::nontype` is presented.

§ 4. Implementation experience

§3.6. Renaming `std::nontype` requires no implementation experience because it is merely changing the name of a symbol.

§3.4. Replacing `std::nontype` with `std::constant_wrapper` was implemented at [GitHub1]. This was done by forking the existing reference implementation of [P0792R14]. This implementation already had `std::move_only_function` and `std::function` constructors taking `std::nontype` and has been thoroughly tested, which made it easy to verify that `std::constant_wrapper` could be used instead. A reference implementation for `std::copyable_function` was also tested for `std::constant_wrapper` integration in that repository.

§3.1. Using a `std::stateless_constructor` tag instead has been implemented at [GitHub2]. This was done by forking the existing reference implementation of [P0792R14].



§ 5. Wording

The following changes are relative to [N5008] with the changes in [P2781R8] applied.

§ 5.1. Option A — Replacing `std::nontype` with `std::constant_wrapper`

§ 5.1.1. [version.syn]

In [version.syn], update the feature-test macro:



```
#define __cpp_lib_copyable_function 202306L 20XXXXL // also in <functional>
#define __cpp_lib_function 20XXXXL // also in <functional>
#define __cpp_lib_function_ref 202306L 20XXXXL // also in <functional>
#define __cpp_lib_move_only_function 202110L 20XXXXL // also in <functional>
```

§ 5.1.2. [utility.syn]

In [utility.syn], delete the declarations of `nontype_t` and `nontype`.



```
// nontype argument tag
template<auto V>
struct nontype_t {
    explicit nontype_t() = default;
};
template<auto V> constexpr nontype_t<V> nontype{};
```

§ 5.1.3. [func.wrap.func] `std::function`

Change the synopsis in [func.wrap.func.general] as follows:



```
namespace std {
    template<class R, class... ArgTypes>
    class function<R(ArgTypes...)> {
    public:
        using result_type = R;

        // [func.wrap.func.con], construct/copy/destroy
        function() noexcept;
        function(nullptr_t) noexcept;
        function(const function&);
        function(function&&) noexcept;
        template<class F> function(F&&);
        template<auto f> function(constant_wrapper<f>) noexcept;
        template<auto f, class U> function(constant_wrapper<f>, U&&) noexcept;
        template<auto f, class T> function(constant_wrapper<f>, T*) noexcept;
```




```

function& operator=(const function&);
function& operator=(function&&);
function& operator=(nullptr_t) noexcept;
template<class F> function& operator=(F&&);
template<class F> function& operator=(reference_wrapper<F>) noexcept;

~function();

// [func.wrap.func.mod], function modifiers
void swap(function&) noexcept;

// [func.wrap.func.cap], function capacity
explicit operator bool() const noexcept;

// [func.wrap.func.inv], function invocation
R operator()(ArgTypes...) const;

// [func.wrap.func.targ], function target access
const type_info& target_type() const noexcept;
template<class T> T* target() noexcept;
template<class T> const T* target() const noexcept;
};

// [func.wrap.func.deduct] deduction guides
template<class R, class... ArgTypes>
    function(R(*)(ArgTypes...)) -> function<R(ArgTypes...)>;

template<class F> function(F) -> function<see below>;
template<auto f> function(constant_wrapper<f>) -> function<see below>;
template<auto f, class T> function(constant_wrapper<f>, T&&) -> function<s
}

```

Make the following changes; specific wording to be provided in the next revision:



TODO: Provide specifications for new constructor overload set and for the deduction guides, with behavior analogous to how `std::nontype` is currently used in `std::function_ref`.

§ 5.1.4. [func.wrap.move] `std::move_only_function`

Change the synopsis in [func.wrap.func.general] as follows:



```

namespace std {
    template<class R, class... ArgTypes>
    class move_only_function<R(ArgTypes...) cv ref noexcept(noex)> {
    public:
        using result_type = R;

        // [func.wrap.move.ctor], constructors, assignment, and destructor
        move_only_function() noexcept;
        move_only_function(nullptr_t) noexcept;
        move_only_function(move_only_function&&) noexcept;

```



```

template<class F> move_only_function(F&&);
template<class T, class... Args>
    explicit move_only_function(in_place_type_t<T>, Args&&...);
template<class T, class U, class... Args>
    explicit move_only_function(in_place_type_t<T>, initializer_list<U>, Args&&...);
template<auto f> move_only_function(constant_wrapper<f>) noexcept;
template<auto f, class U> move_only_function(constant_wrapper<f>, U&&) noexcept;
template<auto f, class T> move_only_function(constant_wrapper<f>, cv T*) noexcept;

move_only_function& operator=(move_only_function&&);
move_only_function& operator=(nullptr_t) noexcept;
template<class F> move_only_function& operator=(F&&);

~move_only_function();

// // [func.wrap.move.inv], invocation
explicit operator bool() const noexcept;
R operator()(ArgTypes...) cv ref noexcept(noex);

// // [func.wrap.move.util], utility
void swap(move_only_function&) noexcept;
friend void swap(move_only_function&, move_only_function&) noexcept;
friend bool operator==(const move_only_function&, nullptr_t) noexcept;

// [func.wrap.move.deduct], deduction guides
template<auto f> function(constant_wrapper<f>) -> function<see below>;
template<auto f, class T> function(constant_wrapper<f>, T&&) -> function<see below>;

private:
    template<class VT>
        static constexpr bool is-callable-from = see below; // exposition only
};
}

```

Make the following changes; specific wording to be provided in the next revision:



TODO: Provide specifications for new constructor overload set and for the deduction guides, with behavior analogous to how `std::nontype` is currently used in `std::function_ref`.

§ 5.1.5. [func.wrap.copy] `std::copyable_function`

Change [func.wrap.copy.class], as follows:



```

namespace std {
    template<class R, class... ArgTypes>
    class copyable_function<R(ArgTypes...) cv ref noexcept(noex)> {
    public:
        using result_type = R;

        // [func.wrap.copy.ctor], constructors, assignments, and destructors
        copyable_function() noexcept;
        copyable_function(nullptr_t) noexcept;
        copyable_function(const copyable_function&);

```



```

copyable_function(copyable_function&&) noexcept;
template<class F> copyable_function(F&&);
template<class T, class... Args>
    explicit copyable_function(in_place_type_t<T>, Args&&...);
template<class T, class U, class... Args>
    explicit copyable_function(in_place_type_t<T>, initializer_list<U>, Args&&...);
template<auto f> copyable_function(constant_wrapper<f>) noexcept;
template<auto f, class U> copyable_function(constant_wrapper<f>, U&&) noexcept;
template<auto f, class T> copyable_function(constant_wrapper<f>, cv T*) noexcept;

copyable_function& operator=(const copyable_function&);
copyable_function& operator=(copyable_function&&);
copyable_function& operator=(nullptr_t) noexcept;
template<class F> copyable_function& operator=(F&&);

~copyable_function();

// [func.wrap.copy.inv], invocation
explicit operator bool() const noexcept;
R operator()(ArgTypes...) cv ref noexcept(noex);

// [func.wrap.copy.util], utility
void swap(copyable_function&) noexcept;
friend void swap(copyable_function&, copyable_function&) noexcept;
friend bool operator==(const copyable_function&, nullptr_t) noexcept;

// [func.wrap.copy.deduct], deduction guides
template<auto f> function(constant_wrapper<f>) -> function<see below>;
template<auto f, class T> function(constant_wrapper<f>, T&&) -> function<see below>;

private:
    template<class VT>
        static constexpr bool is-callable-from = see below; // exposition only
};
}

```

Make the following changes; specific wording to be provided in the next revision:



TODO: Provide specifications for new constructor overload set and for the deduction guides, with behavior analogous to how `std::nontype` is currently used in `std::function_ref`.

§ 5.1.6. [func.wrap.ref] `std::function_ref`

Change [func.wrap.ref.class], as follows:



```

namespace std {
    template<class R, class... ArgTypes>
    class function_ref<R(ArgTypes...) cv noexcept(noex)> {
    public:
        // [func.wrap.ref.ctor], constructors and assignment operators
        template<class F> function_ref(F*) noexcept;
        template<class F> constexpr function_ref(F&&) noexcept;
        template<auto f>

```



```

constexpr function_ref(nontype_t constant_wrapper<f>) noexcept;
template<auto f, class U>
    constexpr function_ref(nontype_t constant_wrapper<f>, U&&) noexcept;
template<auto f, class T>
    constexpr function_ref(nontype_t constant_wrapper<f>, cv T*) noexcept;

constexpr function_ref(const function_ref&) noexcept = default;
constexpr function_ref& operator=(const function_ref&) noexcept = default;
template<class T> function_ref& operator=(T) = delete;

// [func.wrap.ref.inv], invocation
R operator()(ArgTypes...) const noexcept(noex);

private:
    template<class... T>
        static constexpr bool is-invocable-using = see below; // exposition only

    R (*thunk_ptr)(BoundEntityType, Args&&...) noexcept(noex); // exposition only
    BoundEntityType bound-entity; // exposition only
};

// [func.wrap.ref.deduct], deduction guides
template<class F>
    function_ref(F*) -> function_ref<F>;
template<auto f>
    function_ref(nontype_t constant_wrapper<f>) -> function_ref<see below>;
template<auto f, class T>
    function_ref(nontype_t constant_wrapper<f>, T&&) -> function_ref<see below>;
}

```

Change [func.wrap.ref.ctor] as follows:



```

template<auto f> constexpr function_ref(nontype_t constant_wrapper<f>) noexcept;

9 Let F be decltype(f) constant_wrapper<f>::value_type.

10 Constraints: is-invocable-using<F> is true.

11 Mandates: If is_pointer_v<F> || is_member_pointer_v<F> is true, then f
    constant_wrapper<f>::value != nullptr is true.

12 Effects: Initializes bound-entity with a pointer to an unspecified object or null
    pointer value, and thunk_ptr with the address of a function thunk such that
    thunk(bound-entity, call-args...) is expression-equivalent
    ([defns.expression.equivalent]) to invoke_r<R>(f constant_wrapper<f>::value,
    call-args...).

template<auto f, class U>
    constexpr function_ref(nontype_t constant_wrapper<f>, U&& obj) noexcept;

13 Let T be remove_reference_t<U> and F be decltype(f)
    constant_wrapper<f>::value_type.

14 Constraints:

```



- `is_rvalue_reference_v<U&&>` is `false`, and
- `is_invocable_using<F, cv T&>` is `true`.

15 *Mandates*: If `is_pointer_v<F> || is_member_pointer_v<F>` is `true`, then `f constant_wrapper<f>::value != nullptr` is `true`. 16 *Effects*: Initializes *bound-entity* with `addressof(obj)`, and *thunk_ptr* with the address of a function *thunk* such that `thunk(bound-entity, call-args...)` is expression-equivalent (`[defns.expression.equivalent]`) to `invoke_r<R>(f constant_wrapper<f>::value, static_cast<cv T&>(obj), call-args...)`.

```
template<auto f, class T>
constexpr function_ref(nontype_t constant_wrapper<f>, cv T* obj) noexcept;
```

17 Let F be `decltype(f) constant_wrapper<f>::value_type`.

18 *Constraints*: `is_invocable_using<F, cv T*>` is `true`.

19 *Mandates*: If `is_pointer_v<F> || is_member_pointer_v<F>` is `true`, then `f constant_wrapper<f>::value != nullptr` is `true`. 20 *Preconditions*: If `is_member_pointer_v<F>` is `true`, `obj` is not a null pointer. 21 *Effects*: Initializes *bound-entity* with `obj`, and *thunk_ptr* with the address of a function *thunk* such that `thunk(bound-entity, call-args...)` is expression-equivalent (`[defns.expression.equivalent]`) to `invoke_r<R>(f constant_wrapper<f>::value, obj, call-args...)`.

In `[func.wrap.ref.deduct]`, change the deduction guides as follows:



```
template<auto f>
function_ref(nontype_t constant_wrapper<f>) -> function_ref<see below>;

2          Let          F          be          remove_pointer_t<decltype(f)
constant_wrapper<f>::value_type>.

3 Constraints: is_function_v<F> is true.

4 Remarks: The deduced type is function_ref<F>.
```

```
template<auto f, class T>
function_ref(nontype_t constant_wrapper<f>, T&&) -> function_ref<see below>
```

5 Let F be `decltype(f) constant_wrapper<f>::value_type`.

6 *Constraints*:

- F is of the form `R(G::*)(A...) cv &opt noexcept(E)` for a type G, or
- F is of the form `M G::*` for a type G and an object type M, in which case let R be `invoke_result_t<F, T&>`, A... be an empty pack, and E be `false`, or
- F is of the form `R(*) (G, A...) noexcept(E)` for a type G.



§ 5.2. Option B — Renaming `std::nontype` to `std::constant_arg`

In `[version.syn]`, update the feature-test macro:



```
#define __cpp_lib_function_ref 202306L 20XXXXL // also in <functional>
```

In `[utility.syn]`, modify the header synopsis as follows:



```
// nontype constant argument tag
template<auto V>
    struct nontype_t constant_arg_t {
        explicit nontype_t() = default;
    };
template<auto V> constexpr nontype_t<V> nontype{};
template<auto V> constexpr constant_arg_t<V> constant_arg;
```

In `[func.wrap.ref.class]`, replace every occurrence of `nontype_t` with `constant_arg_t`:



```
namespace std {
    template<class R, class... ArgTypes>
    class function_ref<R(ArgTypes...) cv noexcept(noex)> {
    public:
        // [func.wrap.ref.ctor], constructors and assignment operators
        template<class F> function_ref(F*) noexcept;
        template<class F> constexpr function_ref(F&&) noexcept;
        template<auto f> constexpr function_ref(nontype_t constant_arg_t<f>) noe;
        template<auto f, class U> constexpr function_ref(nontype_t constant_arg_t<f>) noe;
        template<auto f, class T> constexpr function_ref(nontype_t constant_arg_t<f>) noe;

        constexpr function_ref(const function_ref&) noexcept = default;
        constexpr function_ref& operator=(const function_ref&) noexcept = default;
        template<class T> function_ref& operator=(T) = delete;

        // [func.wrap.ref.inv], invocation
        R operator()(ArgTypes...) const noexcept(noex);

    private:
        template<class... T>
            static constexpr bool is-invocable-using = see below; // exposition only

        R (*thunk_ptr)(BoundEntityType, Args&&...) noexcept(noex); // exposition only
        BoundEntityType bound-entity; // exposition only
    };

    // [func.wrap.ref.deduct], deduction guides
    template<class F>
        function_ref(F*) -> function_ref<F>;
    template<auto f>
        function_ref(nontype_t constant_arg_t<f>) -> function_ref<see below>;
    template<auto f, class T>
```

```
function_ref(nontype_t constant_arg_t<f>, T&&) -> function_ref<see below>
}
```



In [func.wrap.ref.ctor], replace every occurrence of `nontype_t` with `constant_arg_t`:



```
[...]
template<auto f> constexpr function_ref(nontype_t constant_arg_t<f>) noexcept
[...]
template<auto f, class U>
constexpr function_ref(nontype_t constant_arg_t<f>, U&& obj) noexcept;
[...]
template<auto f, class T>
constexpr function_ref(nontype_t constant_arg_t<f>, cv T* obj) noexcept;
[...]
template<class T> function_ref& operator=(T) = delete;
```

21 Constraints:

- T is not the same type as `function_ref`,
- `is_pointer_v<T>` is `false`, and
- T is not a specialization of `nontype_t constant_arg_t`.

In [func.wrap.ref.deduct], replace every occurrence of `nontype_t` with `constant_arg_t`:



```
[...]
template<auto f>
function_ref(nontype_t constant_arg_t<f>) -> function_ref<see below>;
[...]
template<auto f, class T>
function_ref(nontype_t constant_arg_t<f>, T&&) -> function_ref<see below>;
[...]
```

§ 6. References

[N5008] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>

[P0792R14] Vittorio Romeo, Zhihao Yuan, Jarrad Waterloo. `function_ref`: a type-erased callable reference (2022-02-08)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0792r14.html>



- [P1045R1] *David Stone*. `constexpr` Function Parameters (2019-09-27)
<https://open-std.org/JTC1/SC22/WG21/docs/papers/2019/p1045r1.html>
- [P2472R3] *Jarrad J. Waterloo, Zhihao Yuan*. `make_function_ref` more functional (2022-05-12)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2472r3.html>
- [P2781R8] *Hana Dusíková, Matthias Kretz, Zach Laine*. `std::constant_wrapper` (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2781r8.html>
- [P2841R1] *Corentin Jabot*. Concept and variable-template `template-parameters` (2023-10-14)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2841r1.pdf>
- [P3312R1] *Bengt Gustafsson*. Overload Set Types (2025-04-16)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3312r1.pdf>
- [GitHub1] *Bronek Kozicki*. Reference implementation of function wrappers with `std::constant_wrapper` (2025-06-19)
https://github.com/MFHava/P2548/compare/master...Bronek:P2548:bronek/with_constant_wrapper
- [GitHub2] *Jan Schultke*. Reference implementation of `std::function_ref` with `std::stateless` (2025-06-14)
https://github.com/Eisenwave/nontype_functional